

Databases 2 - exam - September 5, 2025 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

A. Physical DB (12 points)

Table **City** (**CountryName**, **CityName**, **Population**) contains 10K tuples, stored in a primary B+ tree built on the primary key (**CountryName**, **CityName**), consisting of 4 levels and 1.3K leaf nodes.

Table **Event** (**EventID**, **EventName**, **CityName**, **CountryName**, **Date**, **Category**) contains 1M tuples, stored across 5K blocks, in an entry-sequenced sequential structure.

Consider the following query:

```
SELECT *
FROM City JOIN Event ON City.CityName = Event.CityName AND City.CountryName = Event.CountryName
WHERE City.CountryName = 'Italy'
      AND Event.Date BETWEEN '2023-12-01' AND '2024-01-31' // December '23 and January '24
      AND Event.Category = 'New Year Festival';
```

Assume: $\text{Val}(\text{Category}) = 25$; 1% of events occur between '2023-12-01' and '2024-01-31'; $\text{Val}(\text{CountryName}) = 100$. Suppose that the number of cities is uniformly distributed across countries.

Describe an efficient query plan and estimate its cost for each of the following scenarios:

1. No Auxiliary Structures. Evaluate two possible query plans:
 - i) one starting from the **City** table (3 points) and
 - ii) one starting from the **Event** table (3 points).
2. A secondary B+ tree index is built on table **Event** on the **Date** column, with 3 levels and 1.2K leaf nodes. Write a plan that starts from this B+ index. (4 points)
3. Do the previous plans change if the first line of the SQL query becomes **SELECT Event.***? Provide a clear explanation to support your answer. (2 points)

For each scenario,

- Briefly explain the plan (e.g., join method, scan&look up, etc.).
- Clearly explain the number of blocks/leaf nodes that must be retrieved; report all the needed computations to understand the total cost.
- Provide the complete formulas of the costs, even if the plan is not optimal.
- Simplification: ignore page caching and the materialisation of intermediate results.

Solution.

1. Scenario 1 (No auxiliary)

- (a) Scan the leaf nodes of the B+ **City** table, accessing it exploiting "Italy". For each Italian city, lookup on the **Event** table (scan) to compute the join and evaluate the remaining conditions.

Cost = (3 intermediate levels + 1300/100 leaf nodes) + (10K tuple/100 countries) * scan Event 5K

$$= 3 + 13 + 100 * 5K = \mathbf{500016 \text{ I/O}}$$

- (b) Scan **Event** and filter events by date, category, and country

$$5K + (1\% \text{ of } 1M)/25/100 *$$

10K events (1%) are in the dates of interest, there are 25 different categories and 100 countries: 4 events satisfy the conditions. Each matching event requires a lookup in the **City** B+ tree (accessed using "Italy" and the city name of the event) to compute the join. **CountryName**, **cityName** is a primary key, so there will be only one tuple in the **City** table - only 1 leaf node.

$$* (3 \text{ int levels} + 1 \text{ leaf})$$

$$= 5k + 4 * 4 = \mathbf{5016 \text{ I/O}}$$

2. Scenario 2 (Index on Date)

Access **Date** B+ searching for December 1, 2023 and visiting all the leaf nodes up to January 31, 2024. Here, there will be 10K pointers (obtained as 1% of 1M)

2 intermediate levels +

1% leaf nodes (1% of 1,2K) +

10K pointers (1% of 1M) *

For each of the 10K pointers, we access the full event in the primary **Event** structure (cost 1 for each pointer). In the full tuple, we can check the other conditions on **Category** and **CountryName**. Again, we will find 4 events satisfying all the predicates.

Given the pair country-city of these 4 events, we search them in the B+ to compute the join (cost 3 + 1 for each single event):

$$2 + 12 + 10K * 1 + 4 * (3 + 1) = \mathbf{10030 \text{ I/O}}$$

This plan is less efficient than scanning the sequential structure.

3. If the **Event** table defines a foreign key constraint on (**CountryName**, **CityName**), the optimiser can safely assume that every pair in **Event** is guaranteed to exist in **City**. In this case, the join does not filter tuples, and query plans that start from **Event** can terminate without accessing the **City** table.

The plan starting from **Event** in the first scenario would require only **5k I/O**. The plan of the second scenario would become $2 + 12 + 10K * 1 = \mathbf{10014 \text{ I/O}}$.

If such a constraint is not present, all three plans must explicitly perform the join to verify the matching tuples.

B. Concurrency control (10 points)

Consider the following schedule of transactions:

$S = r_1(x), w_4(t), r_5(x), w_1(y), r_2(y), r_3(t), w_5(y), r_4(y), w_2(z), r_3(z), w_3(y)$

Classify S with respect to VSR (in case, listing the view equivalent schedules), CSR, 2PL, Strict 2PL, and TS Multi (with the conventions adopted for TS Multi under Snapshot Isolation, used for the exercises). Justify your answers exhaustively. For 2PL and TS show the result using the tables.

Solution.

Short answers:

CSR: Yes

VSR: Yes (CSR implies VSR) T_1, T_2, T_5, T_4, T_3

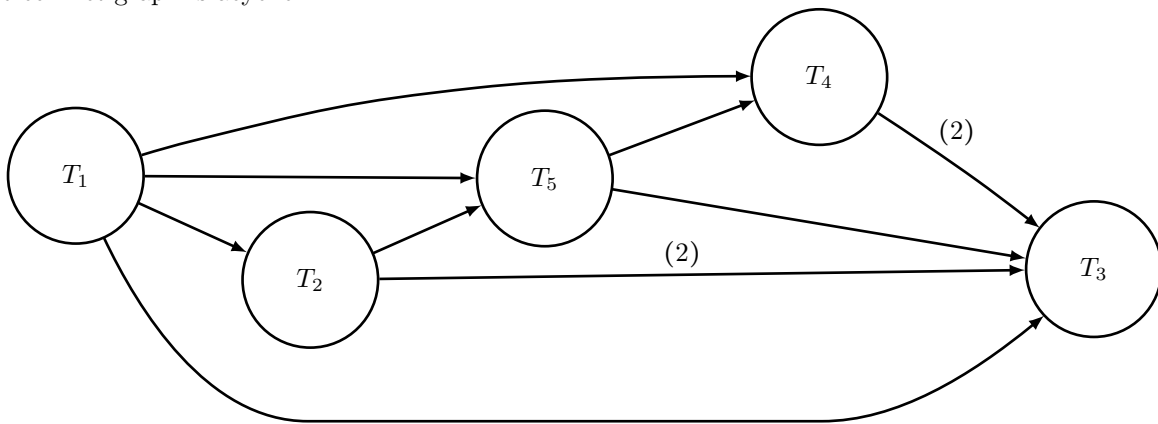
2PL: No T_4 releases t before $r_3(t)$ yet subsequently requests a lock on y for $r_4(y)$, after $w_5(t)$

2PL strict: No (not 2PL)

TS multi: No, because of $r_4(y) - w_3(y)$

Detailed analysis for the CSR class

The conflict graph is acyclic.



Detailed analysis for the VSR class

The schedule is CSR and thus also VSR.

View-equivalent to serial schedule: $r_1(x) w_1(y) r_2(y) w_2(z) r_5(x) w_5(y) w_4(t) r_4(y) r_3(t) r_3(z) w_3(y)$

Detailed analysis for the 2PL class

Transaction 4 must release before $r_3(t)$ and acquire after $w_5(y)$, so the schedule is not 2PL.

	1	2	3	4	5	6	7	8	9	10	11						
t	$\uparrow_4$	w_4	$\downarrow_4$		$\uparrow_3$	r_3					$\downarrow_3$						
x	$\uparrow_1$	r_1	$\uparrow_5$	$\downarrow_1$	r_5		$\downarrow_5$										
y	$\uparrow_1$			w_1	$\downarrow_1$	$\uparrow_2$	r_2	$\downarrow_2$	$\uparrow_5$	w_5	$\downarrow_5$	$\uparrow_4$	r_4	$\downarrow_4$	$\uparrow_3$	w_3	$\downarrow_3$
z					$\uparrow_2$			w_2	$\downarrow_2$	$\uparrow_3$	r_3	$\downarrow_3$					
	1				2	5					3						

Legend

$\uparrow$: read lock request

$\uparrow$: write lock request

$\downarrow$: unlock request

Detailed analysis for the Strict 2PL class

The schedule is not 2PL, so it is not Strict 2PL either.

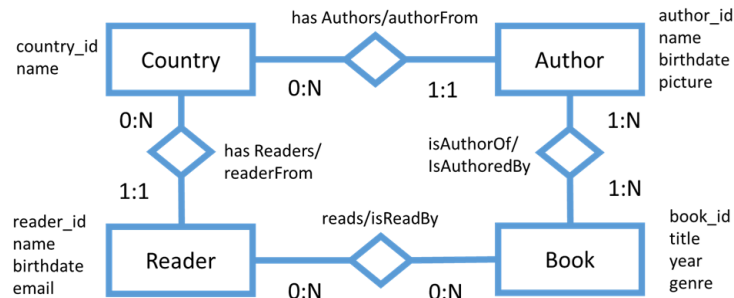
Detailed analysis for the TS Multi class

The schedule is not TS Multi because there are killed transactions.

Operation	RTM(t)	WTM(t)	RTM(x)	WTM(x)	RTM(y)	WTM(y)	RTM(z)	WTM(z)	Killed transactions
$r_1(x)$	0 (0)	0	1 (0)	0	0 (0)	0	0 (0)	0	
$w_4(t)$		0, 4							
$r_5(x)$			5 (0)						
$w_1(y)$						0, 1			
$r_2(y)$					2 (1)				
$r_3(t)$	3 (0)								
$w_5(y)$						0, 1, 5			
$r_4(y)$					4 (1)				
$w_2(z)$								0, 2	
$r_3(z)$							3 (2)		
$w_3(y)$									3 as $3 < \text{RTM}(y) = 4$

C. JPA (10 points)

A digital library application manages information about Books, their Authors, and Readers, as well as the Countries of both authors and readers. Users can explore the catalogue by selecting an author to see the books they have written or first select a country to view the authors associated with it. They can search books by title and see all its data and authors. They can read a selected book and view the list of books they have already read. Library operators can access the list of readers of each book and view the readers' information, their books and country. The expected data scale is as follows: authors are in the order of thousands, countries in the order of tens, books in the order of tens of thousands, readers in the order of millions.



a) Explain the meaning and usage of the annotations `@JoinColumn` and `mappedBy`. (2 points)

With regard to the JPA rendering of the above conceptual schema:

b) Fill in the tables on the next page for **some of** the relationships/directions. In the *Annotation* field, specify `@OneToMany`, `@ManyToMany`, etc., and indicate the `mappedBy` attribute where needed. If 'Required?' is 'No', report in any case the specification of the other properties. Always choose an *owner* (the most reasonable). (4 points)

c) Write the code of the JPA entity that maps the **Author** entity (and its attributes). Take into account the above-mentioned access paths of the application and the cardinalities. Explicitly specify the owner of the relationships, the chosen fetch policies and cascade types: for each aspect, justify and discuss your design choices. (4 points)

Solution.

Answer a) See theory.

In short:

`@JoinColumn`: specifies the FK column that implements the relationship in the database. Normally, it is inserted in the owner entity table.

`mappedBy`: indicates the non-owner side of a bidirectional relationship

Relation: Country → Reader		
	<i>Answer</i>	<i>Motivate your answer</i>
Required?	No	Readers of a country are not requested by the application
Owner	Reader	FK is in reader (country of the reader refers to a country)
FetchType	lazy	given a country, there are several readers
CascadeType	none	readers and countries live independently
Annotation	@OneToMany(mappedBy="country")	

Relation: Reader → Country		
	<i>Answer</i>	<i>Motivate your answer</i>
Required?	Yes	Operators can see the data of each reader, including the country
Owner	Reader	The country of the reader refers (FK) to a country
FetchType	eager or lazy	Eager if the data of the author's country are needed immediately, lazy otherwise
CascadeType	none	countries live independently from readers
Annotation	@ManyToOne @JoinColumn(name="country_id")	

Relation: Book → Author		
	<i>Answer</i>	<i>Motivate your answer</i>
Required?	Yes	When searched by title, for each book the authors are shown
Owner	Author	author chosen as owner
FetchType	eager	reasonable if the author's data are shown immediately with the book data, otherwise could be lazy
CascadeType	none	authors and books can live independently (authors are shared)
Annotation	@ManyToMany (mappedBy="books")	

Relation: Reader → Book		
	<i>Answer</i>	<i>Motivate your answer</i>
Required?	Yes	Readers can see all the books they have read. Operators can see the list of readers of each book.
Owner	Book	book chosen as owner
FetchType	Lazy	Books read by an author are retrieved only when needed
CascadeType	none	books and readers live independently
Annotation	@ManyToMany (mappedBy="readers")	

Answer c)

```
@Entity
public class Author {

    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int author_id;

    private String name;

    @Temporal(TemporalType.DATE)
    private Date birthDate;

    @Lob @Basic(fetch = FetchType.LAZY)
    private byte[] picture;

    @ManyToOne(fetch = FetchType.EAGER) @JoinColumn(name = "country_id")
    private Country country;

    @ManyToMany(mappedBy = "authors", fetch = FetchType.LAZY)
    private List<Book> books;

    // getters and setters
}
```

Owner: Entity Author is the owner of the relationship with Country (it contains the FK to the country); in the code above, Book is chosen as the owner of the relationship with Author (both entities of a ManyToMany relationship could be defined as the owner).

Fetch type: Author→Country: eager (the default) is reasonable, because one author has only one country. Author→Book could also be eager because each author is associated with tens of books.

Cascade type: For Author→Book, do not cascade remove (books can be written by many authors). Persist/merge can be possibly cascaded (in case, add cascade = CascadeType.PERSIST, CascadeType.MERGE to the code). For Author→Country no cascades because countries are shared among many authors.